

Angular Component Lifecycle Hooks

1. What Are Lifecycle Hooks?

- a. Every Angular component has a **lifecycle** — from its creation to destruction.
- b. Lifecycle hooks are **special methods** that allow you to run code **at specific moments** during that lifecycle.
- c. You can think of it like this: A component is “born”, “lives”, “changes”, and is “destroyed” — and Angular gives us hooks to react at each stage.

2. Two Categories of Lifecycle Events

- a. Run once (when component loads or unloads):
 - i. `ngOnInit()` → runs once after the component is initialized.
 1. Commonly used to fetch data or initialize logic.
 - ii. `ngOnDestroy()` → runs once before the component is removed.
 1. Used for cleanup (unsubscribe, remove listeners, etc.).
- b. Run multiple times (when data or inputs change):
 - i. `ngOnChanges()` → runs whenever `@Input()` data changes.
 1. Used to respond to changes in parent-provided data.

3. How to Use a Lifecycle Hook

To use a lifecycle hook in your component:

1. Import the lifecycle interface from Angular.
2. Implement it in your component class.
3. Create the lifecycle method and add your logic inside

✓ **Example:**

```
lifecycle

import { Component, OnInit, OnDestroy } from '@angular/core';

@Component({
  selector: 'app-product-list',
  templateUrl: './product-list.component.html'
})
export class ProductListComponent implements OnInit, OnDestroy {

  ngOnInit() {
    console.log('Component initialized');
    // Fetch data or set up logic here
  }

  ngOnDestroy() {
    console.log('Component destroyed');
    // Cleanup logic here
  }
}
```

Notice the **ng** prefix in the method name (e.g. **ngOnInit**), even though the interface is just **OnInit**.

4. Commonly Used Hooks

Hook	Triggered When	Common Use
ngOnInit()	Component is created	Fetch data, initialize logic
ngOnChanges()	Input properties change	React to data updates
ngOnDestroy()	Component is destroyed	Cleanup, unsubscribe

Event Binding in Angular

1. Property Binding ([])

- Used to bind **data from the component** → **to the template**.
- This is **one-way data flow**, from TypeScript to HTML.
- You bind to **expressions** (they produce a value).

✓ **Example:** `<img [src]="product.imageUrl" /> <!-- src is set by the component's data -->`

2. Event Binding (())

- Used to bind **from the template** → **to the component**.
- Commonly used to respond to **user actions** like clicks, inputs, or submissions.
- You bind to **statements** (they perform an action).
- Syntax: `(event)="statement"`

✓ **Example:**

```
<button (click)="filter = 'Arms'">Arms</button>
<button (click)="filter = 'Heads'">Heads</button>
```

```
<button (click)="filter = ''">All</button>
```

Here, each button **sets the filter property** in the component when clicked.

3. How It Works

- i. `[ ]` = Bind **to a property** (pass data **down**).
- ii. `( )` = Bind **to an event** (send data **up**).
- iii. In the example, clicking a button triggers a change in the component's `filter` property.
- iv. When that property changes, the template updates automatically thanks to Angular's change detection.

4. Key Difference

Direction	Symbol	Used For	Example
Component → Template	<code>[]</code>	Property Binding (expressions)	<code>[src]="image"</code>
Template → Component	<code>()</code>	Event Binding (statements)	<code>(click)="doSomething()"</code>

Showing and Hiding Content with `*ngIf`

Why We Need It

In most applications, we often need to **show or hide elements** based on certain conditions — for example:

- Displaying a login button only if the user is not logged in.
- Showing an error message only if a form is invalid.
- Hiding a loading spinner once data has finished loading.

The `*ngIf` Directive

- `*ngIf` is a **built-in Angular structural directive**.
- It **adds or removes elements from the DOM** based on a condition (Boolean expression).
- Syntax:



- The element and its children are **completely removed** from the DOM when the condition is false.

🧠 **Structural directives** (like `*ngIf`, `*ngFor`, and `*ngSwitch`) modify the structure of the DOM — that's why they start with an asterisk `*`.

Alternatives to `*ngIf`

Hidden Binding

- Binds to the native `hidden` attribute instead of using a structural directive.
- Hides the element **visually**, but it still exists in the DOM.
- Syntax:



Use this when you need the element to stay in the DOM for layout or performance reasons.

`*ngSwitch` Directive

- Works like a JavaScript `switch` statement.
- Used when there are **multiple content blocks** and you want to show only one depending on a

single expression.

- General pattern:



```
<div [ngSwitch]="viewModel">
  <p *ngSwitchCase="'list'">List view</p>
  <p *ngSwitchCase="'grid'">Grid view</p>
  <p *ngSwitchDefault>Default view</p>
</div>
```

Use this when you have multiple possible states to display.

Directive	Type	Removes from DOM?	Common Use
*ngIf	Structural	✓ Yes	Show/hide based on condition
[hidden]	Attribute	✗ No	Temporarily hide while keeping structure
*ngSwitch	Structural	✓ Yes	Toggle between multiple views

Styling in Angular From Encapsulation to ngClass & ngStyle

1. Component-Level Styling and Encapsulation

In Angular, every component can have its own **CSS file** that applies **styles only to that component**.

This is called **style encapsulation**; it prevents styles in one component from accidentally affecting elements in another.

By default, Angular automatically **scopes your styles** to the component's template using unique attributes it generates behind the scenes.

Key idea:

Global styles live in **styles.css**, while component-specific styles live in files like **component-name.component.css**.

This keeps styles modular, predictable, and easy to maintain.

If you only need a few styles for a component, you can also define them directly inside the **styles** property of the **@Component** decorator instead of using a separate CSS file.

2. Class Binding

Angular allows dynamic control over which CSS classes apply to an element.

This is done through **class binding** or **ngClass**.

a. Class Binding

i. Syntax

A code editor snippet with a dark background and light text. It shows the HTML element `<div [class.active]="isActive"></div>` where `[class.active]` is highlighted in green and `"isActive"` is highlighted in yellow. There are three small circles in the top left corner of the editor.

```
<div [class.active]="isActive"></div>
```

- ii. The class `active` is applied only if `isActive` is `true`.

b. `ngClass`

- i. Used when you need to toggle **multiple classes** dynamically.
- ii. Accepts an **object**, where keys are class names and values are Boolean conditions.
- iii.

```
<div [ngClass]="{ active: isActive, disabled:
isDisabled }"></div>
```

- iv. You can also provide an **array** of class names or a **string** containing space-separated classes

Use Case:

Conditionally styling elements (e.g., marking an item as “selected,” “error,” or “highlighted”).

3. Style Binding

Similar to class binding, style binding allows you to change inline CSS styles dynamically.

a. Single Style Binding

```
<div [style.color]="isWarning ? 'red' : 'black'"></div>
```

- i. The element’s text color changes based on the value of `isWarning`.

b. `ngStyle`

- i. Works like `ngClass`, but for inline CSS styles.
- ii. Takes an object where each key is a style property and its value is the style value.

```
<div [ngStyle]="{ color: statusColor, fontSize: '18px' }"></div>
```

Use Case:

Dynamically adjusting styles based on data, such as applying a warning color or highlighting updated content.

🧠 **Note:** Unlike `ngClass`, `ngStyle` does **not** accept arrays or strings – it only works with objects.

4. Using CSS Preprocessors and Frameworks

Angular supports popular styling tools and frameworks out of the box, such as:

- a. Sass (SCSS) – for nested rules, variables, and reusable mixins.
- b. **Bootstrap**, **Tailwind**, and others – for utility-first or pre-styled component design systems.

You can enable Sass when creating a new Angular project with:

```
ng new my-app --style=scss
```

Use Case:

- Sass → cleaner, modular styling with variables and nesting.
- Tailwind → faster UI development using utility classes.
- Bootstrap → ready-made responsive design system.

Summary

Concept	Purpose	Example Syntax	Notes
Encapsulation	Isolate component styles	Component CSS file	Prevents global leaks
Class Binding	Apply a single class conditionally	<code>[class.className]="condition"</code>	Simple conditional styling
ngClass	Apply multiple classes dynamically	<code>[ngClass]="{classA: condA, classB: condB}"</code>	Supports objects, arrays, strings
Style Binding	Bind inline style dynamically	<code>[style.color]="colorVar"</code>	For single styles
ngStyle	Apply multiple styles dynamically	<code>[ngStyle]="{ color: colorVar, fontSize: '16px' }"</code>	Object-only
Sass / Frameworks	Advanced styling or pre-built UIs	SCSS, Tailwind, Bootstrap	Optional but powerful